DISTRIBUTED SYSTEMS

Chapter 4 - Naming in Distributed Systems

4.1 Introduction

Names play a very important role in all computer systems. They are used to share resources, to uniquely identify entities, to refer to locations, and more. An important issue with naming is that a name can be resolved to the entity it refers to.

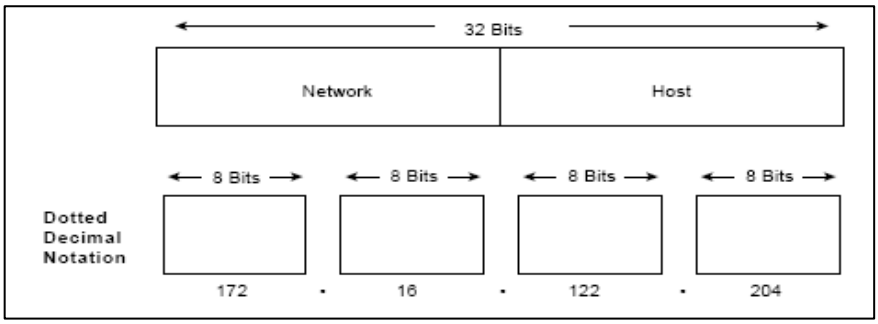
Naming in distributed systems provides a way to identify and locate resources (like files, services, or users) using unique names, decoupling them from their physical locations, and uses "name resolution" to translate these logical names into physical addresses. It involves mapping human-readable (user-friendly), symbolic names (e.g., user@host) to machine-oriented addresses (e.g., IP addresses) to identify and access resources like files, printers, and services. It provides abstraction, separating an entity's identity from its physical location, enabling easy resource sharing, location independence, and hierarchical organization for scalability,

In a distributed system, the implementation of a naming system is itself often distributed across multiple machines. How this distribution is done plays a key role in the efficiency and scalability of the naming system.

4.2. Identifying Resources on the Internet – Recap

Resources (nodes, files, users, website etc) are identified on the internet using;

1) IP Address, a unique 32 bits number/ label assigned to every device/resource connected on a network that uses the Internet for Communication (TCP/IP Network - Transmission Control Protocol/Internet Protocol)
   Example 192.168.10.200

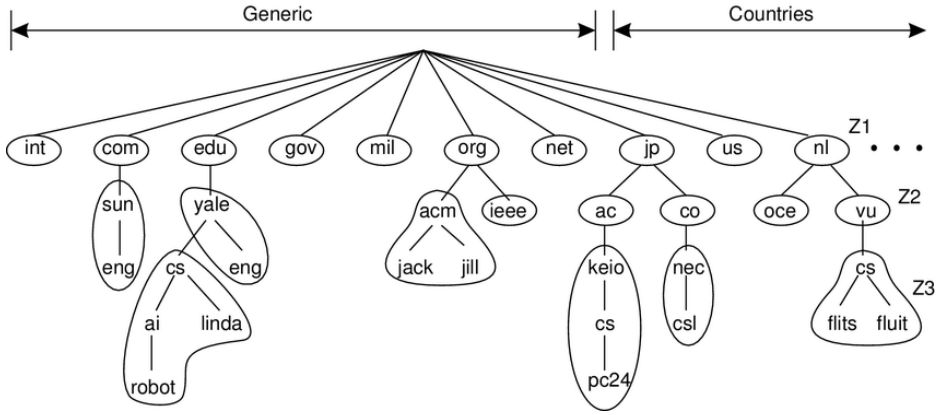


2) Uniform Resource Locator (URL) is the full, specific web address directing to a particular resource on the Internet
   Format **protocol://host/path/filename**
   Exampe. http://www.nos.org/computers/internet/url.htm
3) Domain Name: A domain name is the user-friendly alphanumeric name/identifier, used to identify a one or more IP address (Website)
   Example https://karu.ac.ke/, http://google.com)
   Domain Names are used in the URL - A domain acts as a site's name, whereas the URL acts as the specific pathway

4.2. Key Concepts in Distributed System Naming:

1) **Entity:** An entity in a distributed system can be practically anything. Typical examples include resources such as hosts, printers, disks, and files. Other well-known examples of entities that are often explicitly named are processes, users, mailboxes, newsgroups, Web pages, graphical windows, messages, network connections, and so on.

   Depending on the workload, an entity may be active for any duration between multiple years to only a few seconds. Whatever their lifetime, these entities must be able to communicate with each other.
2) **Name (IP Address):** A name in a distributed system is a string of bits or characters that is used to uniquely identify, locate, and access resources (entities) like files, hosts, or services across a network.
3) **Address**: The name of the access point to an entity. It is used to locate and it enables an entity to be operated. The access point is yet another, but special, kind of entity in a distributed system example IP address, Port#, Telephone # etc
   - An entity can offer more than one access point
   - An entity may have more than one access point/address
   - An entity may change its access points/addresses
   - So using an address as a reference is inflexible and human unfriendly
   - A better approach is to use a name that is location independent, much easier, and flexible to use
4) **Identifier:** A unique name for an entity that does not change. A true identifier is a name that has the following properties
   - An identifier refers to at most one entity.
   - Each entity is referred to by at most one identifier.
   - An identifier always refers to the same entity (i.e., it is never reused).
5) **Name Space**: a structured, logical grouping of names that uniquely identifies resources—such as files, services, or nodes—to prevent naming collisions. It organizes resources into a hierarchy (often a tree graph), allowing systems to map human-readable names to physical network addresses



6) **Name Server**: A name server in a distributed system acts as a specialized repository that translates human-readable names (e.g., example.com) into machine-readable network addresses (e.g., IP addresses). It acts as a directory service, enabling location transparency, allowing users and applications to locate resources without knowing their physical network location

7) **Name System**: A naming system in a distributed system provides a structured mechanism to identify, locate, and access resources (files, services, nodes) across a network. It maps human-readable or identifier-based names to physical network addresses, ensuring location transparency, scalability, and consistent access to shared, often mobile, entities
8) **Name Service**: Mechanism that provides a consistent, uniform, and human-readable way to identify and locate resources (e.g., servers, files, users) across a network, mapping these names to system-internal identifiers like IP addresses.
9) **Name Resolution**: The process of mapping a name to the entity's address or identifier.
10) **Resolution Process**: The mechanism by which a client follows paths through the name space to find the target.
11) **Reverse Name Resolution**: Translating an address back into a human-readable name.

4.3. Name Service

A Naming Service in distributed systems is a specific service whose aim is to provide a consistent and uniform naming of resources, thus allowing other programs or services to localize them and obtain the required metadata for interacting with them. It is a specialized service that acts as a directory, mapping human-readable, symbolic names (e.g., www.example.com or /home/user/file) to machine-readable addresses or identifiers (e.g., IP addresses, object IDs).

It enables resource location, sharing, and lookup across a network, essential for maintaining flexible, location-independent access to services and data.
Example Domain Name System (DNS)

4.3.1. Benefits

1) Resource localization
2) Uniform naming
3) Device independent address (e.g., you can move domain name/website from one server to another server seamlessly).
4) Name Resolution: Translating the Human friendly name to the IP address
5) Publish-Subscribe systems**:** In distributed systems it enables entities to indicate which messages they are interested in receiving. In other words, they subscribe to certain messages with the naming service.
6) Attributes and Bindings**:** Name services store attributes (e.g., type, creation date) along with the address. A binding is the association between a name and its attributes.
7) Caching**:** To enhance performance, frequently used names and their mappings are stored locally, reducing network traffic and latency.

4.3.2. Challenges

1) **Consistency:** Maintaining consistent name-to-address mappings across all replicas is difficult, often requiring a trade-off with performance (weak vs. strong consistency).
2) **Availability:** Name services must be highly available; DNS uses replication across multiple servers to ensure uptime.
3) **Security:** Susceptible to attacks, such as DNS spoofing or DDoS attacks, which can redirect traffic or make the system unavailable.

4.4. Name System

Name system is the concrete, hierarchical network infrastructure that implements name service. A naming system in a distributed system maps human-readable names to machine-readable addresses (IPs), allowing resources to be located, shared, and managed across a network

4.4.1. Benefits

1) Resource Identification and Access: Naming systems allow users and processes to uniquely identify entities (e.g., via URLs or file paths) across the network, acting as a bridge between logical names and physical locations.
2) Location Transparency/Independence: The ability to access a resource without knowing its physical location.
3) Scalability: The ability to handle growth in both the number of resources and geographical distribution.
4) Manage complexity
5) Support for Mobility: Naming systems enable mobile users or migrating processes to maintain access to resources without needing to know the new location of the resource.
6) Name Resolution: The system maps human-readable names to machine-interpretable identifiers or network addresses (e.g., a service name to a specific IP and port).
7) Simplified Communication: Names make it easier for devices and services to communicate effectively.
8) Fault Tolerance: Naming systems can enhance reliability by providing backup and alternative paths to resources.
9) Security: Proper naming systems include mechanisms to verify the authenticity and integrity of resource identifiers.
10) User-Friendly: Intuitive naming systems improve user interaction with the network by simplifying resource access.
1) Efficiency: Efficient naming reduces latency in resource retrieval, improving overall system performance.

Name Service Versus Name System - Key Differences:

Aspect	**Name Service**	**Name System (DNS)**
Definition & Scope	A service that provides mappings between names and attributed, enabling network communication	The global, distributed system responsible for storing, managing, and delivering these name-to-IP mappings
Structure	A general concept or protocol (e.g., Naming Service, Directory Service). Name Service is the protocol used to navigate that structure.	A structured, hierarchical, or distributed database (e.g., root servers, TLDs, Authoritative Name Servers). A Naming System organizes names into a namespace (e.g., hierarchical directories)
Operation	Answers the question "What is the identifier for this name?".	Executes the resolution process by querying various nodes (Nameservers) until the correct IP is found.

Implementation	A name service can be simple	Naming system in a distributed environment must handle scalability, partitioning, and caching, such as DNS
Analogy	Concept of a phone directory	actual, physical phone book with the numbers

4.5. Naming Schema in Distributed Systems

Naming schemes (schema) are the rules by which names are given to individual entities. Naming schemes in distributed systems play a critical role in resource identification and location. By using naming systems, distributed systems can abstract the complexity of resource management and enable seamless interaction between components, regardless of their location or underlying infrastructure.
Types of Names Schemes used in distributed Systems;

1) Flat Naming / Simple Schema: Uses unique identifiers, often for efficiency and location independence.
2) Structured Naming / Hierarchical Naming: Uses human-friendly, hierarchical naming schemes, such as distributed file systems or LDAP.
3) Attribute-Based Naming: Identifies entities by specific attributes rather than fixed paths, providing flexibility in locating specific services or devices

4.5.1. Flat Naming / Simple Naming:

Flat naming in distributed systems assigns unique, unstructured identifiers (long random bit strings) to resources without hierarchical structure, and location information focusing on efficiency, decentralization, and high scalability. Uses unique identifiers (often bit strings) without location information. These identifiers are typically long, ensuring global uniqueness.
Example

1) MAC Address: act as permanent physical addresses for local network communication (LAN), often formatted as six pairs of hexadecimal digits (e.g., 00:1A:2B:3C:4D:5E).
2) A UUID (Universally Unique Identifier) is a 128-bit label used to uniquely identify information in computer systems, often formatted as 32 hexadecimal characters divided into five groups separated by hyphens (e.g., xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx). It ensures uniqueness across distributed systems without a central authority, frequently used for database keys

Features

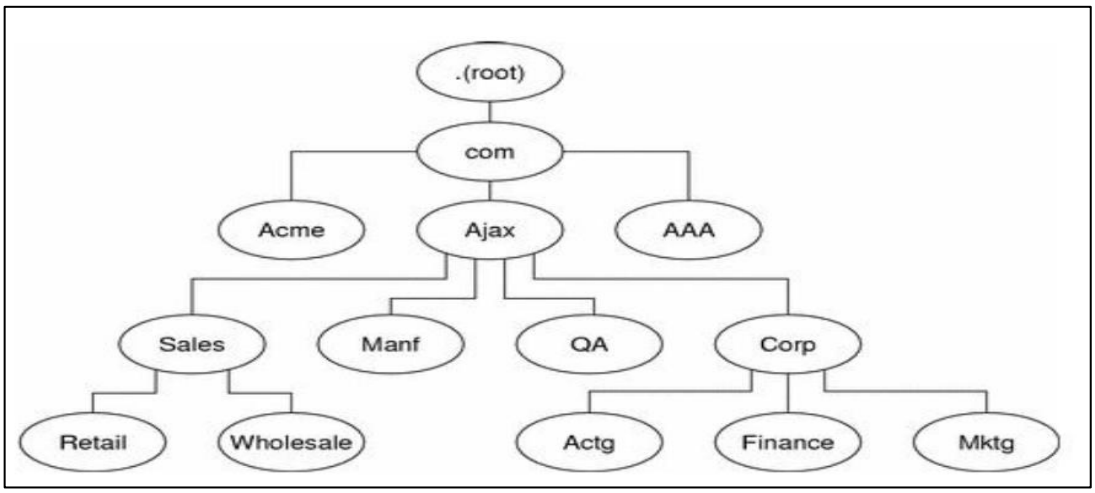
1) Unique Identifiers: Each resource gets a globally unique identifier, preventing conflicts. The Global identifier does not change, making it suitable for identifying entities that move frequently
2) Simplicity: The lack of hierarchy simplifies the naming system, reducing administrative overhead.
3) Flexibility: Flat naming can adapt to various environments without needing structural changes.
4) Decentralization: It supports decentralized systems by not relying on a central authority.
5) Scalability: Flat naming easily scales, accommodating an increasing number of resources.
6) Avoids Bottlenecks: Without central points of failure, flat naming enhances system reliability.
7) Lack of Structure: Names contain no information about the entity's address or location, avoiding the need for rigid structural hierarchy.
8) Naming conflicts as the system grows

9) Name Resolution Approaches: Because the name does not hint at the location, finding the entity requires special techniques, such as
   - Broadcasting/Multicasting: Sending a message to all nodes (or a specific group) asking for the entity ID, often used in local-area networks (LANs).
   - Forwarding Pointers: An entity that moves leaves a pointer at its old location, enabling chains to be followed to the new location.
   - Distributed Hash Tables (DHTs): Nodes are organized into a logical structure (like a ring) and use hashing to locate entities, providing scalability for large-scale systems, such as in Chord.
10) Scalability & Performance: While efficient in small systems or LANs (e.g., ARP), flat naming can become inefficient at scale, often necessitating mechanisms like Distributed Hash Tables (DHTs) to manage resource lookup efficiently

Flat naming is suitable for small-scale distributed systems with a limited number of resources and minimal naming requirements. Flat naming is often seen in file systems, where each file is assigned a unique name within a directory.

4.5.2. Structured / Hierarchical Naming

Structured naming introduces a hierarchical structure to the names of objects or entities. Structured naming in distributed systems organizes resources using human-readable, hierarchical names, often represented as labeled graphs (e.g., directory trees). It organizes objects into logical groupings using a hierarchical naming convention. Each level of the hierarchy represents a different attribute or characteristic of the object, allowing for a more organized and systematic naming approach. Structured naming is commonly used in various domains, including file systems, network addressing, and organizational hierarchies.



In a structured naming scheme, objects are named based on their position within the hierarchy.

Examples

1) **DNS (Domain Name System):** The most prominent example, organizing web resources in a hierarchical, distributed tree structure. DNS is a distributed naming system that translates domain names (e.g., www.example.com) into IP addresses and vice versa

2) **LDAP (Lightweight Directory Access Protocol):** Used to look up users and resources in a structured, tree-like forma. LDAP is a protocol used for accessing and managing directory information. LDAP servers provide a distributed directory service for storing and retrieving hierarchical data such as user accounts, groups, and organizational units. LDAP enables centralized management of directory information across distributed systems, facilitating authentication, authorization, and access control in enterprise environments.
3) **File Systems:** NFS or Unix-like file structures where file names are paths.

Features

1) Hierarchy Structure: Names are organized logically, frequently using paths similar to file systems (e.g., /home/user/file or sub.example.com).
2) Graph-Based Namespace: The namespace is structured as a directed graph, containing directory nodes (which hold naming tables) and leaf nodes (which represent the actual entities).
3) Name Resolution: The process of translating a human-readable name into a physical address by traversing the graph, often starting at a root node.
4) Directory Nodes: These store pointers to other nodes or entities, organizing the namespace into subtrees.
5) Separation of Duties: Higher-level directory nodes store location records, while leaf nodes (low-level domains) represent the actual entity locations
6) Path Names: These are sequences of labels for navigation, such as absolute paths (starting from the root) or relative paths.

Benefits

1) Human-Readable: Easier to remember than numerical IP addresses.
2) Scalability: Allows decentralized control where different authorities manage subtrees (e.g., DNS zones).
3) Flexibility: Supports easy organization of resources by category, location, or department.
4) Allows for easier organization, navigation, and identification of objects.
5) It also provides a clear relationship between objects within the hierarchy
6) Scalability
7) Efficient Lookup and management

Challenges / Issues

1) Name Space Complexity: Managing large-scale graphs requires careful design to prevent bottlenecks.
2) Alias Management: Handling multiple names for the same entity.
3) Network Proximity: Ensuring efficient lookups based on where the request originates.

4.5.3. Attribute – Based Naming:

Attribute-based naming assigns names to objects based on their specific attributes or characteristics. Also known as **Directory services**, where the entities have a set of associated attributes that can be used for searching. Instead of relying on a hierarchical structure, this scheme uses descriptive attributes to uniquely identify objects. Each object is assigned a name that reflects its relevant attributes, such as its type, purpose, or other distinguishing characteristics. Attribute-based naming allows for more expressive and meaningful names.

This naming scheme is often used in systems where the attributes of objects play a crucial role in their identification and categorization. For example, in a database, tables may be named based on the entities they represent or the data they store. In a software development context, functions or variables may be named based on their purpose or the data they manipulate.

Features /Components of Attribute Naming

1) **Attributes:** Describe aspects of the entity, including type, location, or owner.
2) **Directory Service:** A distributed database or registry that stores these descriptions.
   Directory services are centralized databases that store, manage, and organize network resources, user identities, and attributes to enable authentication, authorization, and information retrieval. Directory services form the foundation of modern identity and access management (IAM), a core capability of enterprise IT systems. They function as a "phonebook" for IT environments, using protocols like LDAP for fast, read-intensive access to data, such as users, groups, devices, and policies.
3) **Flexibility Querying:** Enables partial matches and complex queries, unlike rigid hierarchical naming systems.
4) **Search Capabilities:** Unlike hierarchical or flat naming, users can locate entities by specifying a subset of known attributes, making it ideal for dynamic environments.
5) **Identifiers:** In some systems, attributes are hashed to create a unique identifier for locating the entity in a peer-to-peer network (e.g., Chord), such as hash(type=printer)

Common Directory Service Technologies

1) **Active Directory (AD):** Developed by Microsoft, it is the most common enterprise directory service, utilizing LDAP (Lightweight Directory Access Protocol) for managing user identities and resources.
2) **LDAP Directories:** Open, standard protocol used for accessing and managing directory information.
3) **Cloud Directories:** Services like Azure Active Directory or AWS Directory Service provide directory services, allowing organizations to manage identities in cloud-native or hybrid environments.
4) **Network Information Service (NIS):** An older, specialized directory service, often used in Unix environment

Benefits

1) Simplified authentication procedures: Allow users to authenticate once and use a token or ticket to establish their identity going forward.
2) Automated access control: Directory services allow administrators to centralize and automate their approach to permissions and roles
3) Strong encryption and compliance: Modern directory services are equipped with some of the strongest encryption tools available, ensuring that communication and resource sharing remain safe and reducing the likelihood of a data breach.
4) High availability: Directory services are designed to process millions of authentication requests at the same time without affecting their performance

5) Flexibility with modern cloud environments**:** Directory services can be easily integrated into on-premises and cloud-based environments, leveraging both physical and virtual resources and blending them seamlessly.
6) Enhanced Security**:** Strengthens security through consistent application of policies and access controls.
7) Resource Location**:** Enables users to easily find and access printers, applications, and other network resources

4.6. Naming in Distributed Systems Key Requirements & Functions:

1) Location Transparency**:** The name does not reveal the physical location of the entity.
2) Location Independence**:** An entity's name does not change when its location changes.
3) Scalability**:** Naming systems must handle large numbers of entities and lookups.
4) Caching**:** Improves performance by storing frequently used name-to-address mappings.

4.7. Name Resolution - Domain Name System (DNS)

Name resolution in distributed systems is the process of mapping human-readable, logical names (e.g., URL or service name) to physical network addresses (e.g., IP addresses) to locate resources, files, or services. It uses distributed databases most notably DNS to handle large-scale lookups, ensuring scalability, fault tolerance, and efficient resource access across multiple nodes.

4.7.1. Key Mechanisms and Types

1) Hierarchical Naming **(e.g., DNS):** Organizes names into a tree-like structure, where different levels are managed by different servers, allowing for efficient, delegated resolution.
2) Distributed Hash Tables **(DHTs):** Used for flat names, mapping identifiers to nodes in a decentralized, peer-to-peer fashion, often providing lookup efficiency.
3) Attribute-Based Naming**:** Entities are resolved by matching their required attributes rather than a specific name, allowing for more flexible resource discovery.
4) Centralized Naming**:** A central server maintains all mappings, which is simple but risks becoming a bottleneck or single point of failure.
5) Directory Services**:** Used in corporate networks (e.g., LDAP) to store user and device information.
6) File System Naming**:** Systems like NFS (Network File System) map file paths to server locations.

4.7.2. Key Components

1) **Name Space:** A graph consisting of directory nodes (grouping names) and leaf nodes (representing resources).
2) **Name Server:** Stores the bindings between names and network addresses.
3) **Resolution Process:** The mechanism by which a client follows paths through the name space to find the target.
4) **Reverse Name Resolution:** Translating an address back into a human-readable name.

Common Examples

5) **DNS (Domain Name System):** The primary system for translating domain names to IP addresses on the Internet.

6) **WINS (Windows Internet Name Service):** A legacy system for mapping NetBIOS names to IP addresses in Microsoft networks.
7) **P2P Systems:** Utilize techniques like Chord to manage distributed, flat address spaces.
8) Effective name resolution is critical to ensure that resources can be located even if their physical location changes, enhancing the overall flexibility of a distributed system

4.8. Domain Name System

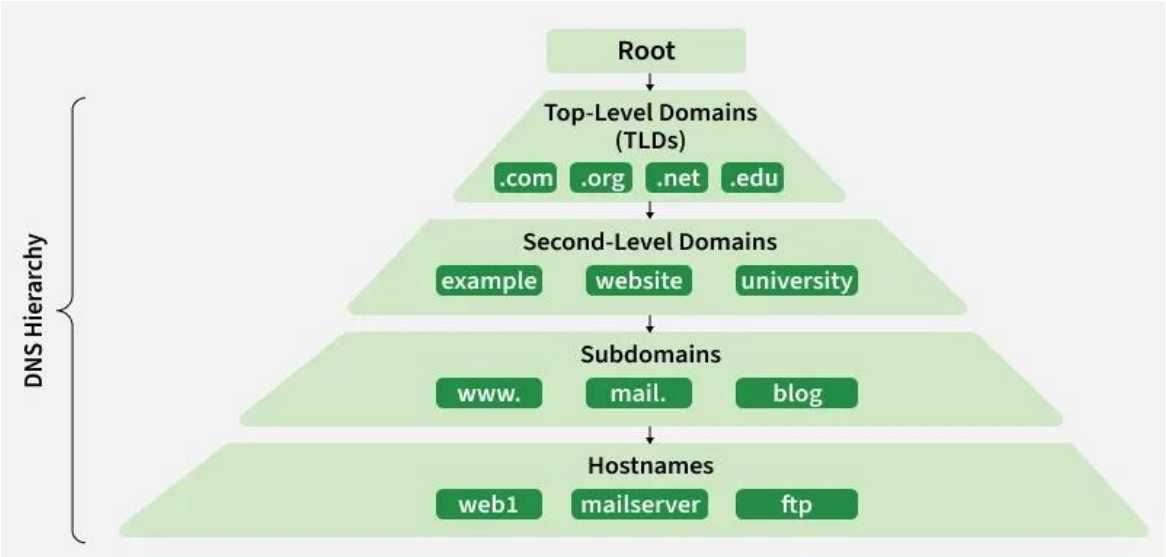
The Domain Name System (DNS) is a hierarchical and distributed naming system that translates human-readable domain names into IP addresses, enabling users to access websites easily.

1) Converts domain names (e.g., www.geeksforgeeks.org) into IP addresses.
2) Eliminates the need to remember numerical IP addresses.
3) Works using a hierarchical structure of servers.
4) Uses root, TLD, and authoritative DNS servers for resolution.
5) Supports caching to improve speed and reduce network traffic.

4.8.1. Structure of DNS

DNS operates through a hierarchical structure, ensuring scalability and reliability across the global internet infrastructure. Here's how it's organized:

1) **Root DNS Servers:** These are the highest-level DNS servers and know where to find the TLD servers. They are crucial for directing DNS queries to the correct locations.
2) **TLD Servers:** These servers manage domain extensions like .com, .org, .net, .edu, .gov and others. They help route queries to the authoritative DNS servers for specific domains.
3) **Authoritative DNS Servers:** These are the servers that store the actual DNS records for domain names. They are responsible for providing the correct IP addresses that allow users to reach websites.

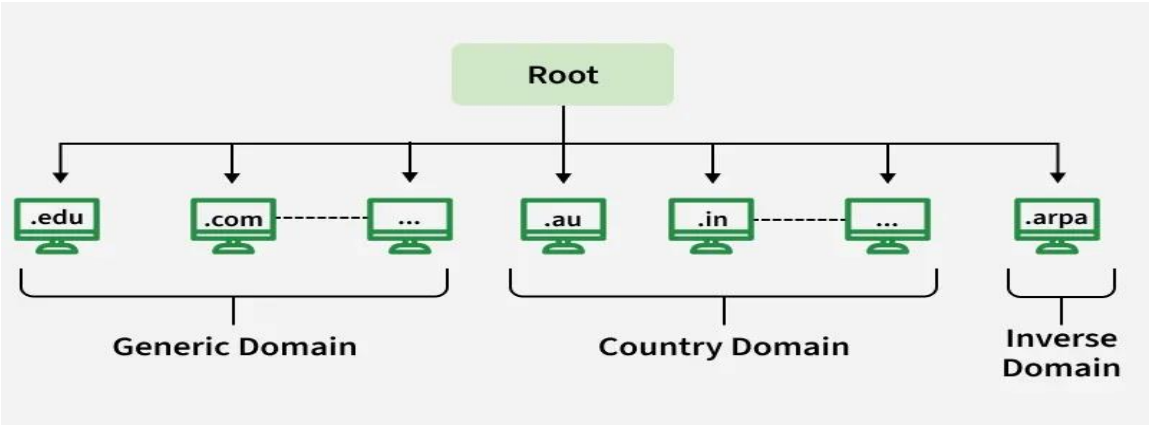


4.8.2. Types of Domains

DNS helps manage a wide variety of domain types to organize the vast number of websites on the internet. Here are the primary categories:

1) **Generic Domains:** These include top-level domains like .com, .org, .net and .edu. These are widely used and recognized across the world.
2) **Country Code Domains:** These domains represent specific countries or regions, such as .in for India, .us for the United States, .uk for the United Kingdom and .jp for Japan.

3) **Inverse Domains:** Used for reverse DNS lookups, these domains help map IP addresses back to domain names. Reverse DNS lookups are useful for diagnostics and security purposes, ensuring that the source of network traffic is legitimate. So DNS can provide both the mapping for example to find the IP addresses of geeksforgeeks.org then we have to type



The client machine sends a request to the local name server, which, if the root does not find the address in its database, sends a request to the root name server, which in turn, will route the query to a top-level domain (TLD) or authoritative name server.

- The root name server can also contain some hostName to IP address mappings.
- The Top-level domain (TLD) server always knows who the authoritative name server is.
- So finally the IP address is returned to the local name server which in turn returns the IP address to the host.

4.8.2. DNS Lookup – Name Resolution

DNS Lookup, also called DNS Resolution, is the process of translating a human-readable domain name into its corresponding IP address. The process involves:

1) **DNS Resolver:** Initiates the lookup process. Also called a DNS client.
2) **Recursive Query:** Resolver queries multiple servers on behalf of the client until the IP is found.
3) **Iterative Query:** Resolver asks servers for the best answer available.
4) **Non-Recursive Query:** Resolver queries a server that already has the record in its cache.

How it works – Details

1) **User Input:** You enter a website address (for example, www.geeksforgeeks.org) into your web browser.
2) **Local Cache Check:** Your browser first checks its local cache to see if it has recently looked up the domain. If it finds the corresponding IP address, it uses that directly without querying external servers.
3) **DNS Resolver Query:** If the IP address isn't in the local cache, your computer sends a request to a DNS resolver. The resolver is typically provided by your Internet Service Provider (ISP) or your network settings.
4) **Root DNS Server:** The resolver sends the request to a root DNS server. The root server doesn't know the exact IP address for www.geeksforgeeks.org but knows which Top-Level Domain (TLD) server to query based on the domain's extension (e.g., .org).
5) **TLD Server:** The TLD server for .org directs the resolver to the authoritative DNS server for geeksforgeeks.org.

6) **Authoritative DNS Server:** This server holds the actual DNS records for geeksforgeeks.org, including the IP address of the website's server. It sends this IP address back to the resolver.
7) **Final Response:** The DNS resolver sends the IP address to your computer, allowing it to connect to the website's server and load the pa**ge.**

4.8.3. DNS Caching and TTL (Time-to-Live)

1) DNS caching is a mechanism that stores DNS records locally to avoid querying external DNS servers repeatedly for the same information. This speeds up the browsing experience and reduces network traffic.
2) TTL (Time-to-Live) is the amount of time that a DNS record is cached before it expires. When the TTL expires, the cache is cleared and a fresh DNS query must be made. The TTL is defined by the authoritative DNS server, which can adjust this based on the nature of the domain.
   Example: If the TTL for www.geeksforgeeks.org is 3600 seconds, the record is cached for 1 hour before a fresh lookup is needed.

4.8.4. Reverse DNS Lookup

Reverse DNS Lookup is the process of mapping an IP address back to its corresponding domain name. This is the opposite of the typical DNS lookup, where a domain name is resolved to an IP address. Reverse DNS is commonly used for:

1) **Network Diagnostics:** System administrators use reverse DNS to determine the domain name associated with a specific IP address. This helps identify the source of network traffic.
2) **Email Security:** Many email servers perform reverse DNS lookups to verify that incoming emails are coming from legitimate sources. This helps prevent spam and fraud.